Order Flow Software

Architecture, Security & Commercial Product Design

Full Design & Strategy Discussion

1. Production Code — Scope & Guarantees

There is no absolute guarantee of production-level code from any AI system — but the following outlines what can be delivered and the path to production readiness.

1.1 Core Architecture

Tick Data Engine (C++)

- Lock-free ring buffers for tick ingestion (SPSC/MPSC queues)
- Market microstructure processing: bid/ask aggregation, trade classification (Lee-Ready algorithm)
- Order flow metrics: Delta, Cumulative Delta, Volume Profile, Footprint charts, Imbalance detection
- High-performance time-series storage: memory-mapped files, kdb+, or TimescaleDB integration

Indicator Computation Engine

- Plugin/strategy API using shared libraries (.so/.dll) with well-defined C ABI
- Template-based indicator framework — users implement a single interface
- Configurable via YAML/TOML/JSON config files (tick window, volume thresholds, imbalance ratios)

External API Layer

- REST API (cpp-httplib or Crow) for historical queries and config
- WebSocket API (uWebSockets or Boost.Beast) for real-time streaming
- FlatBuffers/Protobuf serialization for low-latency binary protocol

- Authentication via API key / JWT

1.2 Code Quality Assessment

Dimension	What Is Delivered	Gap to Close
Architecture	Production-grade patterns	Review and adapt to your infra
Correctness	High quality, tested patterns	Write and run test suites
Performance	Optimized patterns	Profile on actual hardware
Error Handling	Comprehensive coverage	Stress test with malformed data
Thread Safety	Lock-free where applicable	Verify under concurrency load
Exchange Compliance	Generic implementation	Add venue-specific protocol

---

2. Testing Strategy

For order flow software, all three testing layers are required — and the sequence matters critically.

Layer 1 — Unit & Component Testing

Test each module in isolation before touching market data.

- Ring buffer correctness — overflow, underflow, concurrent reads/writes
- Order flow calculations — Delta math, imbalance ratios, VWAP formulas with known inputs
- Config parser — malformed YAML, missing fields, boundary values
- API serialization — Protobuf/FlatBuffers encode-decode round trips

Tools: Google Test (gtest), Google Benchmark, Catch2

Layer 2 — Historical Tape Data

This is the most valuable and safest testing layer for order flow logic — fully deterministic, contains real market microstructure events, and allows unlimited regression testing.

- Footprint chart construction accuracy
- Cumulative delta across full sessions
- Volume profile correctness at known price levels
- Imbalance detection on known high-activity periods (FOMC releases, open/close)

Data Sources

Source	Type	Notes

CME Group	Historical tick data	Paid, authoritative
Rithmic / CQG	Historical tick feed + full depth	Full DOM history
Databento	Normalized historical tick data	Developer-friendly
Your broker	DOM/tick history export	Most provide this
Binance / Coinbase	Historical trade data	Free, crypto-focused

Layer 3 — Simulated / Synthetic Data

Used to test edge cases that historical data may never contain.

- Zero-volume bars and locked/crossed markets
- Massive single-print imbalances
- Tick storms (100,000+ ticks/second burst)
- Clock skew and out-of-order timestamps
- Partial fills, cancelled orders mid-sequence
- Flash crash scenarios

Layer 4 — Market Replay

Feed historical data through the live pipeline at real-time speed. The application processes it identically to live data — validating WebSocket streaming, API latency, and config hot-reload under realistic timing.

Layer 5 — Live Paper Trading

- Connect to broker simulation environment (Rithmic Paper, IB Paper, CME Globex test)
- Tests full integration: tick ingestion → processing → API output → indicator display

Recommended Testing Sequence

Phase	Activity	Timeline
1	Unit tests (gtest)	Day 1–3
2	Historical tape replay	Week 1–2
3	Synthetic stress tests	Week 2–3
4	Market replay at real-time speed	Week 3–4
5	Paper trading / broker sim env	Week 4–6
6	Live data — restricted hours	Week 6+
7	Full production	After 30 days live stability

The Single Most Important Test

Timestamp integrity under load. Order flow analysis is entirely dependent on tick sequencing. Specific test:

- Inject 500,000 ticks/second through the pipeline
- Deliberately introduce 0.1% out-of-order timestamps
- Verify sequence correction logic handles it without corrupting delta calculations

---

3. Commercial Product Architecture

This software is a commercial ISV product targeting multiple trading platforms, sold on a subscription basis to end users. This changes the architecture significantly.

3.1 Overall System Architecture

```
NinjaTrader (C#)  ──▶  Thin C# Wrapper DLL  ──▶  C++ Core Engine (DLL)
Sierra Chart      ──▶  ACSIL C++ study      ──▶  C++ Core Engine (DLL)
TradingView       ──▶  WebSocket bridge     ──▶  C++ Engine API
Bookmap           ──▶  JNI Java bridge      ──▶  C++ Core Engine (DLL)
TT Platform       ──▶  TT SDK adapter       ──▶  C++ Core Engine (DLL)
```

The C++ core engine never changes — only thin adapters are added per platform. This is the core competitive moat.

3.2 NinjaTrader Integration (Phase 1)

NinjaTrader addons are written in C# (.NET). The bridge pattern uses P/Invoke to call into the C++ engine:

- C++ core is compiled, obfuscated, and license-locked
- C# wrapper is minimal — no business logic lives there
- Users only see the C# wrapper — IP stays in the C++ binary

3.3 Platform Scalability Roadmap

Phase	Platform	Integration Method	Timeline
1	NinjaTrader 8	C# P/Invoke → C++ DLL	Now
2	Sierra Chart	Native C++ ACSIL study	3 months
3	TradingView	WebSocket → Pine Script bridge	6 months
4	Bookmap	JNI → C++ engine	9 months
5	TT / CQG / Rithmic	FIX/native SDK adapters	12 months
6	Web Dashboard	React + WebSocket API	18 months

4. Security Architecture

Commercial software requires multi-layer protection to prevent unauthorized access, cracking, and license bypass. Five layers are required.

Layer 1 — Code Obfuscation & Anti-Reverse Engineering

- OLLVM (Obfuscator-LLVM) — control flow flattening, instruction substitution
- Themida / WinLicense — industry standard for Windows DLL protection
- VMProtect — virtualizes critical code sections (license check, core algorithm)
- Strip all debug symbols from release builds — never ship PDB files

Layer 2 — License Engine (Hardware-Bound)

Each license is cryptographically tied to the end user's machine using a hardware fingerprint:

- CPU ID (via CPUID instruction)
- Disk volume serial number
- Primary NIC MAC address
- Motherboard ID (WMI query)
- Windows installation ID

Fingerprint is SHA-256 hashed and validated against your license server on every startup. Encrypted token cached locally for offline grace period.

Layer 3 — Subscription Enforcement

Tier	Features	Billing
Trial	14 days, limited indicators	Free
Monthly	Full access to all indicators	Monthly recurring
Yearly	Full access + priority support	Annual — discounted
Enterprise	Multi-seat + custom indicators + raw API	Custom pricing

All features are gated by tier at runtime. License token is RSA-2048 signed by your server — cannot be forged.

Layer 4 — API Security (External Interface)

- API Gateway with rate limiting per subscription tier
- DDoS protection via Cloudflare or AWS WAF
- JWT tokens (RS256, short-lived 15-minute expiry)
- API Keys (long-lived, scoped permissions per user)
- Read-only access to stats — no write access to core engine
- Full audit log of every API call

Layer 5 — Runtime Integrity Checks

- Binary hash verification at startup — detect if DLL has been patched
- Anti-debug detection (IsDebuggerPresent, remote debugger, NtQueryInformationProcess)
- Periodic integrity re-checks during runtime — not only at startup
- Automatic shutdown and license revocation on tamper detection

---

5. Subscription & Billing Infrastructure

5.1 Recommended Stack

Component	Technology	Reason
Payment processor	Paddle (primary)	Handles VAT, global tax compliance automatically
Payment processor	Stripe (alternative)	More control, but manual tax handling required
License server	Go or Node.js on AWS/GCP	Fast, scalable, battle-tested
Database	PostgreSQL	Reliable, ACID-compliant for billing records
License validation	RSA-2048 signed tokens	Cannot be forged offline

5.2 Lifecycle Event Handling

- payment_succeeded → activate license immediately
- payment_failed → 3-day grace period, then suspend
- subscription_ended → deactivate license on expiry date
- refund_issued → immediate license revocation
- chargeback → blacklist hardware fingerprint hash

---

6. Build Roadmap

Week	Deliverable
Week 1–2	Core C++ engine + NinjaTrader C# adapter
Week 3–4	License engine + hardware fingerprinting module
Week 5–6	License server backend (Go/Node) + Stripe/Paddle integration
Week 7–8	Public REST + WebSocket API with JWT auth

Week 9–10	Binary obfuscation + anti-tamper integration
Week 11–12	Beta testing with trusted NinjaTrader users
Month 4+	Sierra Chart adapter + load/scale testing
Month 6+	TradingView WebSocket bridge
Month 9+	Bookmap and TT/CQG/Rithmic adapters
Month 18+	Web dashboard + full platform coverage

---

7. Key Architectural Decisions

Where Does the Engine Run?

Option	Description	Recommendation
A — Local DLL	Runs on user's machine. Lowest latency, harder to protect IP, no internet required.	Good for NinjaTrader Phase 1
B — Cloud-hosted	User streams from your server. Easiest IP protection, internet required.	Best for web/API consumers
C — Hybrid	Local engine for speed, cloud for license validation and stats aggregation.	Industry standard — recommended

Option C (Hybrid) is the industry standard approach used by Bookmap and Jigsaw Trading. The local engine provides low-latency processing while the cloud layer handles licensing, telemetry, and API serving.

End of Document